

Relazione del progetto ”Turbo Chess”

Giacomo Fantini
Brando Fabbri
Luca Bevilacqua

20 marzo 2026

Indice

1	Analisi	3
1.1	Descrizione e requisiti	3
1.1.1	Requisiti funzionali	4
1.1.2	Requisiti non funzionali	4
1.2	Modello del Dominio	4
2	Design	6
2.1	Architettura	6
2.2	Design dettagliato	8
2.2.1	Giacomo Fantini	8
2.2.2	Brando Fabbri	13
2.2.3	Luca Bevilacqua	16
3	Sviluppo	22
3.1	Testing automatizzato	22
3.2	Note di sviluppo	22
3.2.1	Giacomo Fantini	22
3.2.2	Brando Fabbri	24
3.2.3	Luca Bevilacqua	25
4	Commenti finali	26
4.0.1	Giacomo Fantini	26
4.0.2	Brando Fabbri	26
4.0.3	Luca Bevilacqua	27
4.1	Difficoltà incontrate e commenti per i docenti	27
4.1.1	Giacomo Fantini	27
4.1.2	Brando Fabbri	28
4.1.3	Luca Bevilacqua	28
5	Guida utente	29
5.0.1	Avvio del gioco	29

5.0.2	Organizzazione cartella risorse	29
5.0.3	Creazione di un pezzo custom	29
5.0.4	Funzionamento del Movimento	32
5.0.5		33
5.0.6	Regole Avanzate	33
5.0.7	Creazione di un Loadout	34
5.0.8	Visualizzare e usare un Loadout	35
6	Esercitazioni di laboratorio	36
6.0.1	giacomo.fantini5@studio.unibo.it	36
6.0.2	luca.bevilacqua6@studio.unibo.it	36

Capitolo 1

Analisi

1.1 Descrizione e requisiti

Il software, creato dalla "Turbo Chess Organization", si pone come obiettivo quello di creare una nuova esperienza del gioco degli scacchi. Questo sarà raggiunto dando la possibilità ai giocatori di poter creare nuovi pezzi con i propri movimenti.

Di base, scacchi è un gioco a turni strategico a due persone dove ogni giocatore dispone di 16 pezzi ed ognuno è in grado di muoverne uno a turno (tramite regole di movimento prestabilite) con anche la possibilità di mangiare pedine dell'avversario. L'obiettivo è quello di fare "Scacco Matto" (Check-mate) all'avversario, ovvero di mettere sotto attacco il Re dell'avversario senza che quest'ultimo sia in grado di difenderlo.

Data l'aggiunta di pezzi custom da parte degli utenti, sarà presente una funzionalità di "loadout" dove ciascun giocatore può scegliere quali schierare e come disporli sulla scacchiera, rispettando però le regole originali. Inoltre si vuole dare la possibilità in futuro di aggiungere nuovi tipi di entità posizionabili sulla scacchiera che, a determinate caratteristiche, possono eseguire azioni che modificano lo stato della partita. Infine il gioco dovrà provvedere varie funzionalità di "Quality of Life" come un sistema di replay direttamente in gioco che permette di ripercorrere l'intera partita ed un sistema di salvataggio che permette di salvare e ricaricare una partita in un secondo momento.

1.1.1 Requisiti funzionali

- La versione riproposta di scacchi deve essere pertinente al gioco originale, in termini di regole.
- L'applicazione dovrà coordinare correttamente le diverse scene del gioco.
- Il gioco dovrà disporre di possibili estensioni da parte dell'utente come PowerUp, Pezzi custom e Loadout.
- Il sistema di replay deve permettere di scorrere la partita senza modificarne lo stato.
- L'applicazione permette la compatibilità con i principali OS.

1.1.2 Requisiti non funzionali

- Il gioco deve presentarsi intuitivo per aiutare i giocatori nuovi a questo tipo di gioco.
- L'applicazione non deve risultare difficile da navigare nel menu e tra le varie schermate.

1.2 Modello del Dominio

La Partita in sé deve aver accesso ad un Gestore di Turni, in modo da poter mandare avanti la partita e una Scacchiera con i Pezzi racchiusi a loro volta per avere un riferimento concreto. Il Gestore di Turni è a conoscenza, e di conseguenza impone, le regole avanzate di scacchi nella sua logica. In base allo stato (contesto) della partita il Gestore di Turni è in grado di associare il corretto comportamento per eseguire il turno. Il sistema di movimento deve poter essere il più generico possibile per poter permettere agli utenti di creare dei nuovi pezzi che possano potenzialmente avere qualunque tipo di movimento (che però abbia senso all'interno di una scacchiera) senza rendere il tutto troppo astruso. La scacchiera deve prevedere il poter contenere tipi di entità diverse (non solo classici pezzi) in modo da permettere l'aggiunta di altre componenti. Data la possibilità di personalizzare i pezzi, l'utente deve poterli equipaggiare mantenendo la stessa struttura e lo stesso peso del loadout classico.

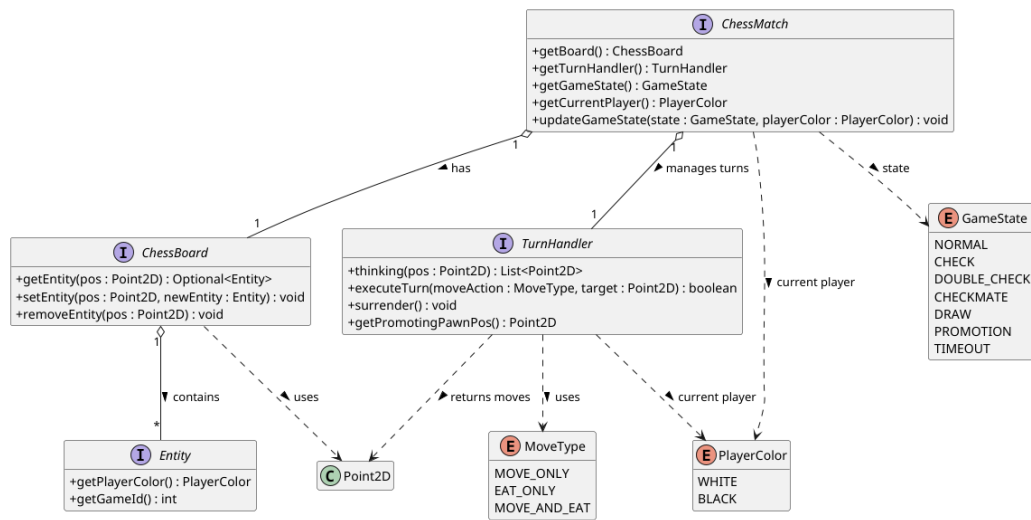


Figura 1.1: Schema UML dell'analisi del dominio, con rappresentate le entità principali ed i rapporti fra loro

Capitolo 2

Design

2.1 Architettura

L'architettura di Turbo Chess segue il pattern architetturale MVC con l'aggiunta di un "GameCoordinator". Il coordinator è l'entry point del programma in quanto presenta sia metodi che permettono lo switch tra scene (operazioni che verranno opportunamente delegate ad un componente apposito che gestisce solamente la view) sia metodi che eseguono funzioni di "setup" prima che la partita sia stata creata, come la creazione e il caricamento da salvataggio della stessa. Il "GameController" è invece il componente che interagisce maggiormente con il model, in quanto crea tutti gli oggetti necessari per l'esecuzione di una partita ed implementa i metodi necessari a svolgere funzionalità specifiche del gioco come la promozione. Il "Chess-Match" è l'oggetto che racchiude tutte le componenti fondamentali del model che, come suggerisce il nome, costituiscono una partita. L'entry point della view è invece la "ViewFactory" che definisce tutti i metodi necessari per cambiare le scene. Sono presenti inoltre vari "observer" che permettono alla view di osservare e reagire visualmente a ciò che accade sulla scacchiera. Gli osservatori sono stati implementati da zero e non usando quelli forniti da specifiche librerie grafiche in modo da non legare il model alla view. In questo modo sostituire in blocco la view necessita solamente di re-implementare una nuova "ViewFactory", di re-implementare i controller specifici delle funzionalità della view scelta e di collegare correttamente gli observer. Inoltre è possibile espandere le possibilità grafiche, come aggiungere effetti grafici particolari, semplicemente aggiungendo un nuovo osservatore che reagisce in modo diverso ad un cambiamento sulla scacchiera.

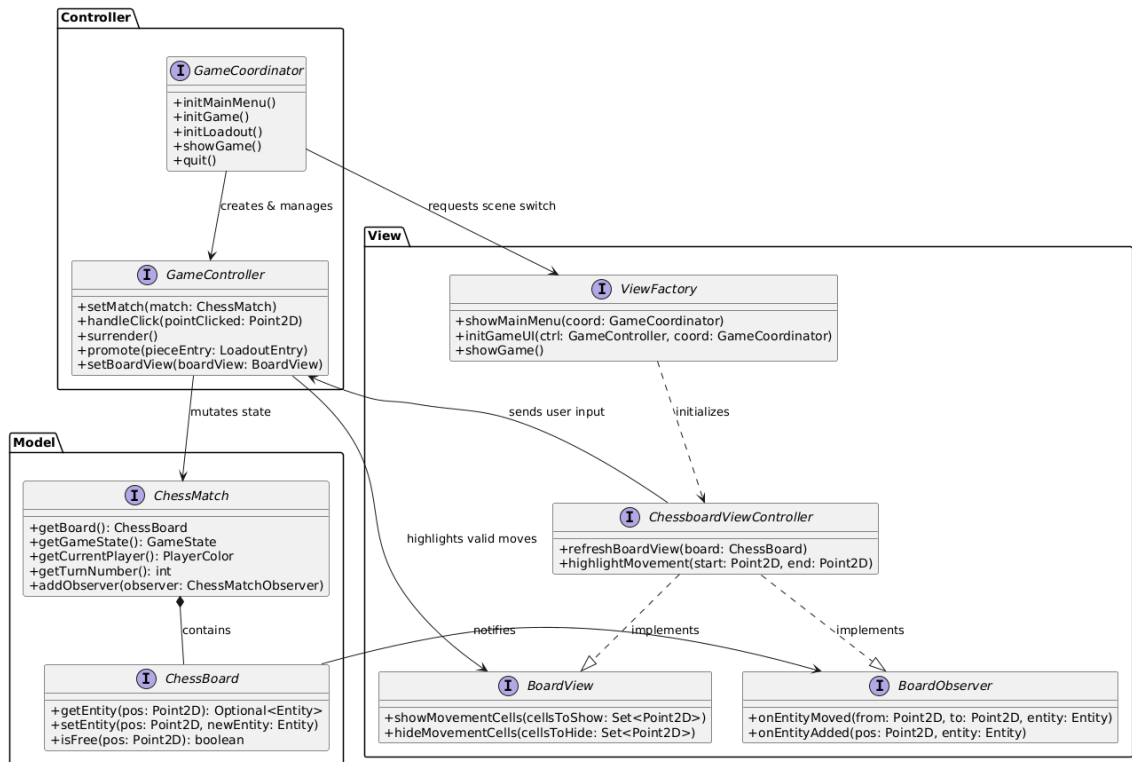


Figura 2.1: Schema UML dell'architettura, con rappresentati i principali attori del pattern MVC

2.2 Design dettagliato

2.2.1 Giacomo Fantini

Astrarre il Movimento

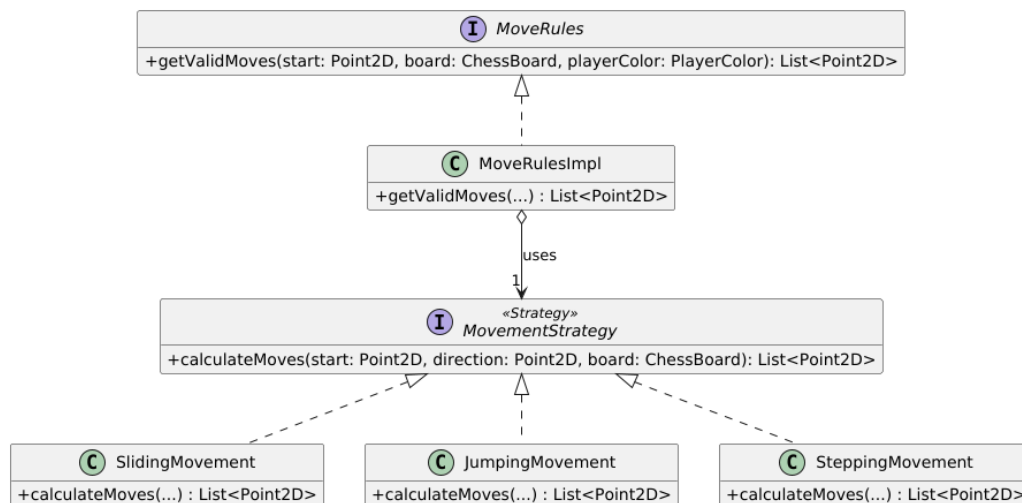


Figura 2.2: Rappresentazione UML del pattern Strategy utilizzato per il calcolo del movimento

Problema Negli scacchi le pedine hanno varie regole di movimento che presentano molti casi particolari. Per rendere il gioco estendibile dagli utenti il modo di gestire le regole di movimento deve essere il più astratto possibile e deve essere semplice per l'utente creare pezzi con set di movimenti custom.

Soluzione L'architettura del sistema sfrutta il **pattern Strategy** per disaccoppiare la generazione delle mosse dalla validazione delle regole di gioco. Il calcolo delle celle di destinazione *grezze* è delegato all'interfaccia **MovementStrategy**, declinata in tre implementazioni concrete che gestiscono le varianti di movimento: **SlidingMovement**, **JumpingMovement** e **SteppingMovement**. La classe **MoveRulesImpl** opera come "contesto": tramite composizione, invoca la strategia per ottenere la lista iniziale delle mosse e vi applica tutti i filtri specifici (verificando, ad esempio, se la cella è libera, occupata da un avversario, o se il pezzo ha già effettuato mosse). Questa netta separazione delle responsabilità rende il sistema altamente estensibile e pronto ad accogliere facilmente nuovi filtri o regole in futuro.

Entità Intercambiabili

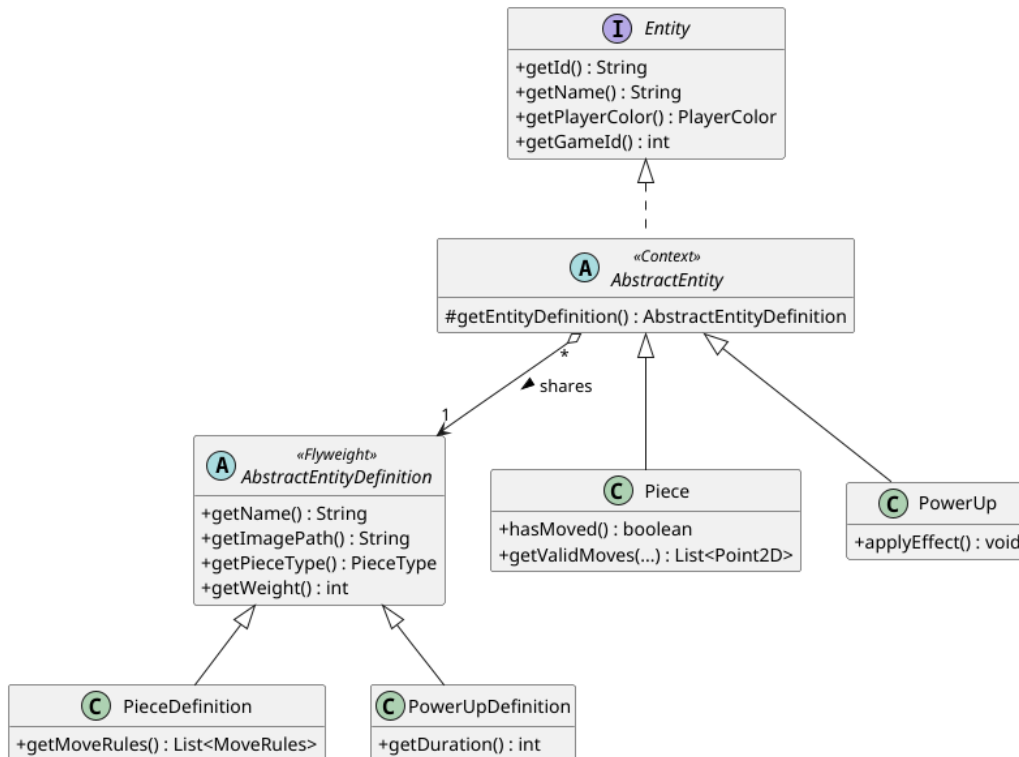


Figura 2.3: Rappresentazione UML della gerarchia di entità posizionabili su una scacchiera

Problema Il gioco deve prevedere la futura presenza di varie **entità** sulla scacchiera, non solo semplici pezzi.

Soluzione La prima soluzione pensata era quella di creare una semplice interfaccia **Entity** che rappresentasse una qualunque entità generica che può essere piazzata su una scacchiera. Questa interfaccia sarebbe poi stata implementata da una classe astratta **AbstractEntity** che avrebbe racchiuso tutte le caratteristiche comuni a varie entità, e poi ci sarebbero state le varie implementazioni concrete come **Piece**. Questo design però presentava un grosso problema di efficienza in quanto ci possono essere più entità con le stesse informazioni "base" su una scacchiera (un pedone ha sempre stesso nome, stessa immagine ecc...), che sarebbero quindi state salvate svariate volte inutilmente. Per questo motivo per ottimizzare la gestione della memoria e disaccoppiare al massimo la logica strutturale da quella di stato, si è deciso di implementare il **pattern Flyweight**, come mostrato in

Figura 2.3 Lo stato *intrinseco* (ovvero le proprietà immutabili e condivise tra pezzi dello stesso tipo) è stato incapsulato nella gerarchia delle classi **AbstractEntityDefinition** (che funge da Flyweight). La definizione astratta contiene tutte le caratteristiche comuni tra varie *entità*, mentre le specializzazioni di questa classe come **PieceDefinition** contengono le caratteristiche specifiche di un tipo di entità, come le regole di movimento per i normali pezzi. Lo stato *estrinseco* (le proprietà contestuali alla partita della singola entità, come il colore di appartenenza, l'ID della pedina nella partita in corso) è invece mantenuto dalla gerarchia **AbstractEntity** (che funge da contesto). In questo modo si garantisce la modularità di entità che possono essere presenti sulla scacchiera e l'ottimizzazione di occupare inutilmente memoria con dati duplicati.

Caricamento e creazione di risorse custom

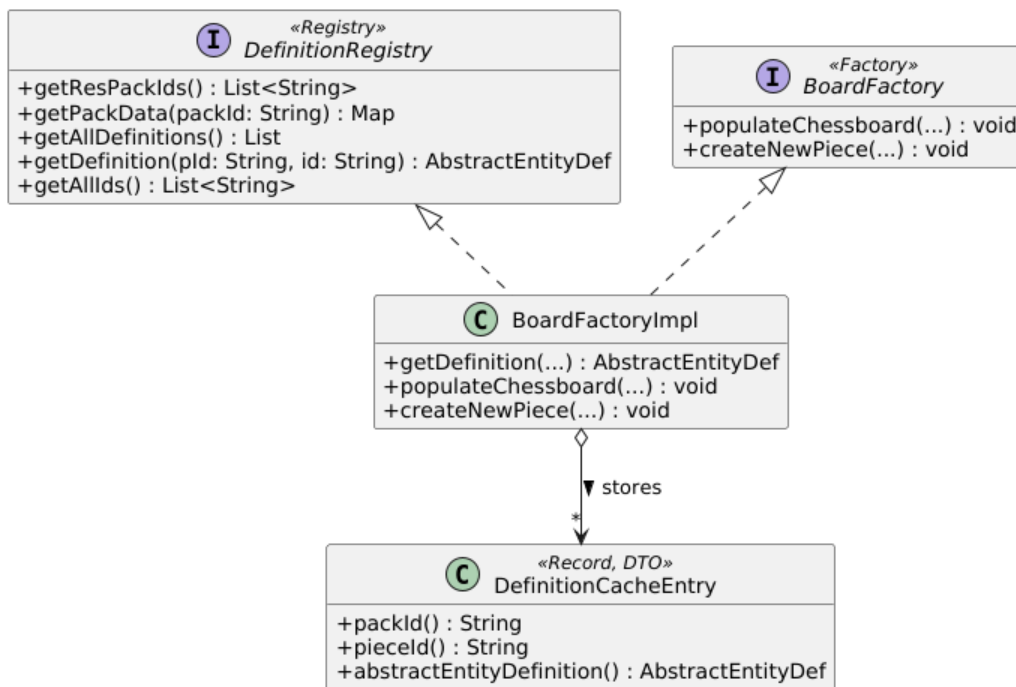


Figura 2.4: Unione dei pattern Factory e Registry per disaccoppiare l'accesso alle definizioni dalla creazione delle entità a runtime

Problema Il sistema deve supportare entità di gioco (pedine) personalizzabili e caricate dall'esterno, le quali devono essere istanziate dinamicamente a runtime (ad esempio durante il popolamento iniziale dei *loadout* o a seguito di eventi particolari come una promozione). Poiché le definizioni di tali

entità sono complesse, è necessario disaccoppiare le definizioni dalla logica di generazione delle pedine vere e proprie sulla scacchiera. Si vuole evitare che la logica di creazione dipenda da operazioni di I/O.

Soluzione La problematica è stata affrontata unendo i design pattern *Registry* e *Factory*, supportati dall'uso di un *Data Transfer Object (DTO)*. La soluzione viene implementata dalla classe `BoardFactoryImpl`, che implementa le due interfacce principali di questo processo:

- **DefinitionRegistry (Pattern Registry):** la sua funzione è quella di un "catalogo" per tutte le definizioni caricate dalla memoria. Per isolare completamente il dominio dalle operazioni su file system, il modulo preposto all'I/O cede i dati al Registry impacchettandoli nell'oggetto `DefinitionCacheEntry`, che agisce da DTO. Il Registry espone poi metodi efficienti per recuperare tali definizioni tramite gli identificatori `packId` e `pieceId`. I dati passati tramite le varie entry possono essere salvati in svariati modi che dipendono dalla specifica implementazione.
- **BoardFactory (Pattern Factory):** incapsula la logica specifica di istanziazione delle pedine e il loro posizionamento sulla `ChessBoard`.

Quando la factory riceve la richiesta di creare un pezzo, essa interroga se stessa (tramite l'interfaccia Registry) per ottenere la corretta definizione che ha salvato internamente, la combina con le informazioni di stato a runtime (posizione, colore) e genera l'entità. In questo modo si nascondono completamente dettagli di creazione e caricamento al resto dell'applicazione.

Gestione dell'immutabilità nelle entità di gioco

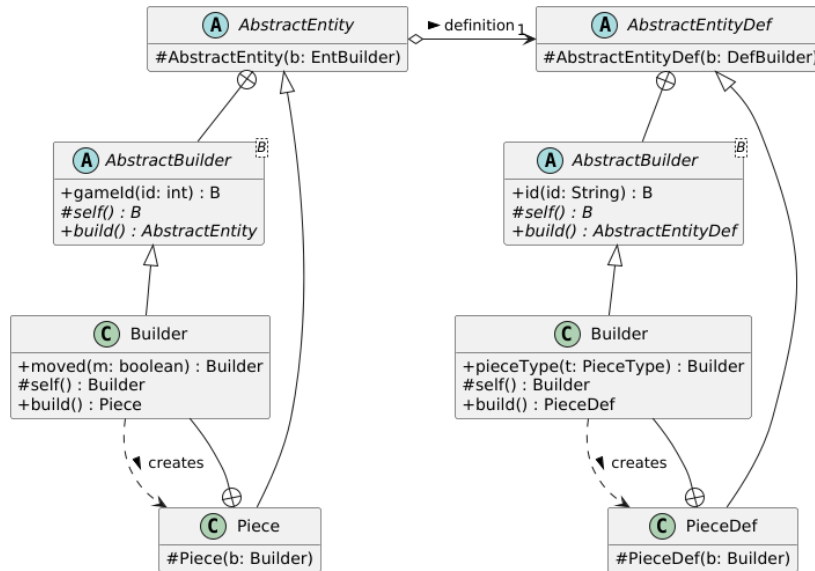


Figura 2.5: Utilizzo del Builder gerarchico per la creazione di entità immutabili

Problema Le entità di base del gioco, come ad esempio i pezzi (*Piece*), sono oggetti con un elevato numero di parametri di inizializzazione che devono poter essere istanziati sia da file (nel caso delle *AbstractEntityDefinition* oppure da Factory nel caso dei *Piece*).

Soluzione Per risolvere il problema si è adottato il **Pattern Builder**. Tutte le entità (come *AbstractEntity* e la sua specializzazione *Piece*) dichiarano i propri campi come *final* o non espongono alcun modificatore di stato, affidando la fase di costruzione ad una classe interna statica chiamata *Builder*. Poiché vi è una forte gerarchia di ereditarietà tra le entità, si è reso necessario implementare anche una gerarchia tra i Builder (*AbstractBuilder* esteso, ad esempio, da *Builder* in *Piece*). In questo si evita di riempire il costruttore con molti parametri, cadendo nell'antipattern del **costruttore telescopico** e si riesce allo stesso a garantire la correttezza dei dati potendo implementare controlli nei vari metodi del builder.

Questa soluzione rispetta il vincolo dell'immutabilità ed è integrabile con varie strategie di creazione di oggetti. I builder concreti delle varie entità possono essere sfruttati dalla factory discussa precedentemente, mentre i builder delle classi che implementano le definizioni possono essere usate anche da

librerie di parsing di file per creare gli oggetti "definitions" che verranno poi salvati.

2.2.2 Brando Fabbri

Gestore dei Turni

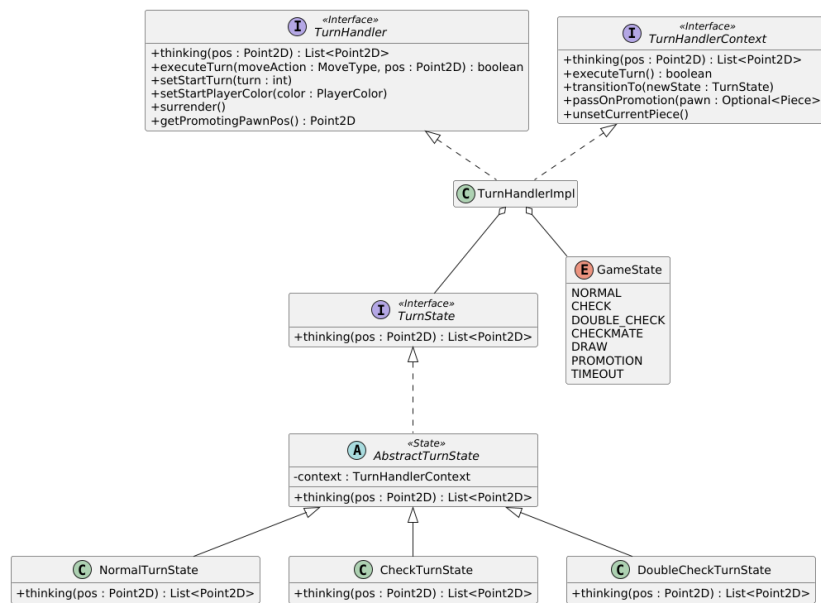


Figura 2.6: Doppia interfaccia per interagire con il TurnHandler in base al proprio ruolo e unione concettuale dei pattern Strategy e State per gestire il comportamento

Problema Durante lo sviluppo del TurnHandler, il quale usava fin dall'inizio un *pattern Strategy* per gestire i diversi comportamenti in base allo stato del gioco, è risultato necessario astrarlo ulteriormente, in modo che il codice stesso fosse più pulito e che rispettasse meglio la modularità del progetto.

Soluzione Per renderlo ancora più facile da modificare o estendere in caso di modding con GameState custom, si è deciso di renderlo affine ad un *pattern State*, ma modificandolo leggermente da renderlo simile all'originale *pattern Strategy*. Infatti seguendo la logica dello State si sarebbe dovuto ridefinire ogni volta il metodo **executeTurn()** del TurnHandler per poi cambiare lo state con **transitionTo()**. Ad ogni turno viene passata un'implementazione diversa di TurnState in base allo stato del gioco e ogni implementazione dispone di un riferimento al TurnHandler tramite l'interfaccia

TurnHandlerContext, che espone solo i metodi getter/setter con cui devono lavorare per evitare che possano chiamare quelli sbagliati.

Regole avanzate e Utility

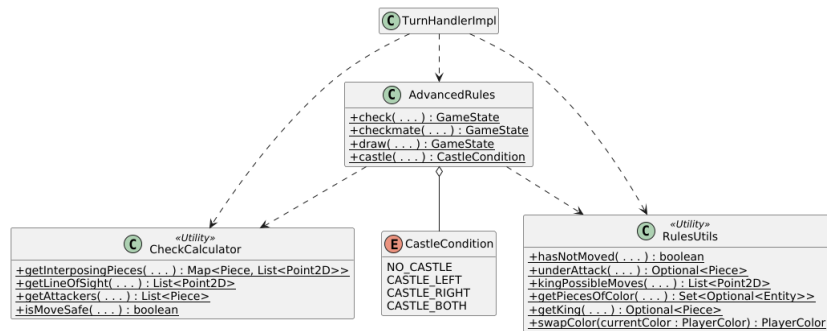


Figura 2.7: TurnHandleImpl usufruisce di tutte le classi utility, di cui sono mostrati i metodi statici

Problema Per implementare le regole e farle rispettare durante il gioco è necessario che siano facilmente accessibili, anche per classi che non siano TurnHandler o le varie implementazioni dei TurnState.

Soluzione Si è deciso di implementare **AdvancedRules** (che contiene le regole avanzate) e **RulesUtils** (che contiene metodi di utility generale per controlli e manipolazione di pezzi) come classi aventi solo metodi statici, in modo da renderle facilmente accessibili ovunque. Con lo sviluppo è saltata fuori anche l'eccessiva complessità del metodo **checkmate()**, e si è deciso di creare un'ulteriore classe utility **CheckCalculator** che dispone di metodi che servono per il calcolo dello scacco matto.

Promozione

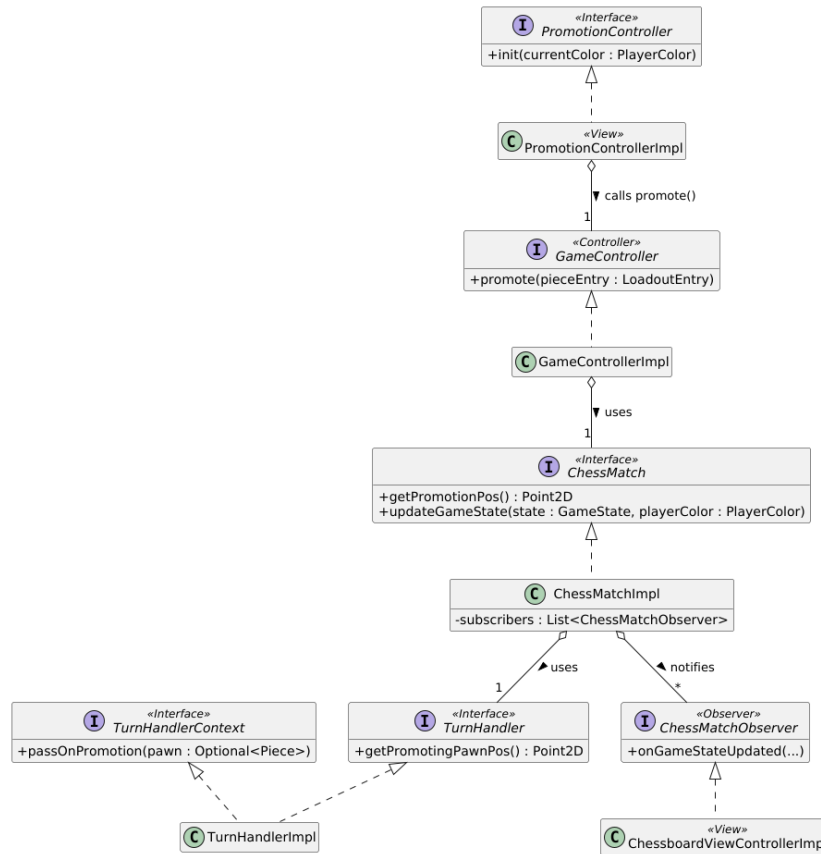


Figura 2.8: La connessione (semplificata) tra le classi per gestire la promozione, Model, Controller e View

Problema La promozione richiede che il giocatore scelga un nuovo pezzo tra quelli presenti nella sua originale collezione, che comporta il dover fermare temporaneamente il gioco finché non viene scelto un pezzo.

Soluzione Si è deciso di dividere il più possibile la parte di codice necessario, facendo in modo che il **Controller** sapesse ridirezionare la **View** non appena riceveva un segnale dal **ChessMatchObserver** nel **Model**, solo che una promozione in certi casi può portare ad uno scacco, controllo che veniva effettuato correttamente ma senza far aggiornare la **View**. Per questo è stato opportuno apportare modifiche al **GameController** in modo che si riferisse ad **AdvancedRules** per poter aggiornare la **View** in base al vero **GameState** del gioco, poiché la promozione avviene e finisce durante uno stesso turno.

Modifica e Creazione di Loadout

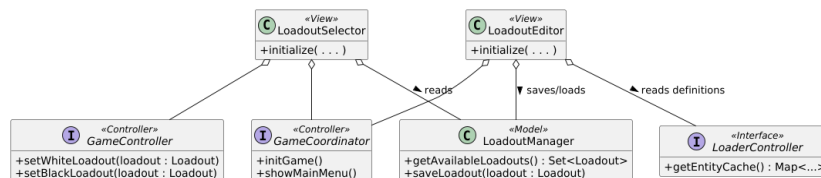


Figura 2.9: Le View di LoadoutSelector e LoadoutEditor e le loro dipendenze

Problema Il sistema richiede un'interfaccia che permetta la mappatura dinamica dei pezzi di scacchi. La View deve gestire la selezione tra pezzi classici e pezzi personalizzati, garantendo che ogni pezzo sia associato correttamente al proprio ID e alle proprietà definite nel file .json del loadout.

Soluzione Invece di fare un'unica schermata, sono stati creati due UI Controller apposti per integrare il funzionamento garantito da **LoadoutManager**.

- **LoadoutEditor**: Visualizza a lato e permette di selezionare singolarmente tutti i pezzi salvati in memoria, sia quelli di scacchi base, sia quelli custom; dispone una griglia in basso in cui aggiungere i pezzi selezionati; permette di salvare il loadout appena creato controllando che sia stato inserito un nome valido e che **isValid()** di **Loadout** non fallisca.
- **LoadoutSelector**: Visualizza a lato e permette di selezionare singolarmente tutti i loadout salvati in memoria; dispone di un bottone "Load" che carica i pezzi contenuti nel loadout selezionato e di un bottone "Use" che salva il loadout selezionato per il giocatore bianco se la checkbox **forBlack** non è spuntata, per il nero se lo è.

2.2.3 Luca Bevilacqua

Sistema di Replay e Logging delle mosse

Problema Il progetto include funzionalità estese rispetto agli scacchi tradizionali, tra cui l'impiego di **pezzi custom** indipendenti dalle regole base. Di conseguenza, i sistemi di replay e di logging non possono assumere un set fisso di comportamenti "standard", ma devono rimanere corretti anche in presenza di entità non previste dagli scacchi classici e di dinamiche aggiuntive (ad esempio spawn/despawn non riconducibili a una semplice cattura).

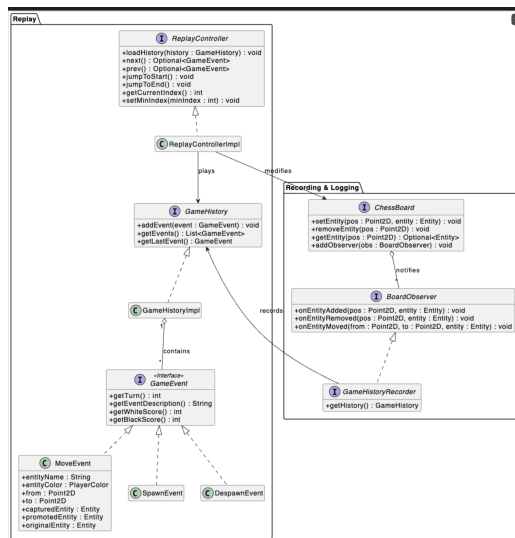


Figura 2.10: completa gestione e controllo del replay e del logging

Soluzione La soluzione sfrutta il design pattern **Observer**: la scacchiera (**ChessBoard**) si limita a notificare, tramite callback, le variazioni del proprio stato (spawn, move, despawn) agli osservatori registrati, senza occuparsi direttamente della persistenza o della costruzione della cronologia. In questo modo ho implementato un componente dedicato (**GameHistoryRecorder**) che, osservando la board via **BoardObserver**, intercetta in tempo reale tutte le mutazioni rilevanti e le traduce in una cronologia di eventi (**GameHistory/GameEvent**). Tale cronologia viene poi riutilizzata in due direzioni:

- **Logging**: ogni evento espone una descrizione testuale (`getEventDescription()`) che alimenta il log delle mosse senza imporre vincoli sulle regole del pezzo.
- **Replay**: il replay controller tratta gli eventi come comandi riproducibili e annullabili, applicandoli o revertendoli su una board dedicata per ottenere navigazione step-by-step (avanti/indietro) senza ricalcolare la “legalità” delle mosse.

Il risultato è un’architettura più pulita e modulare: la board resta concentrata sulle operazioni di gioco, mentre registrazione e riproduzione vengono gestite esternamente. Questo riduce l’accoppiamento, facilita l’estensione e rende il sistema compatibile anche con configurazioni non standard, inclusi i pezzi custom.

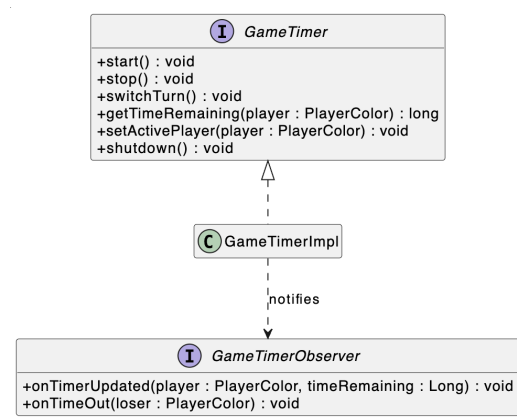


Figura 2.11: Timer

Timer di partita

Problema Il sistema deve gestire una partita a tempo, mantenendo per ciascun giocatore un conteggio aggiornato del tempo rimanente e mostrando tali informazioni in modo reattivo nella UI. Il timer deve inoltre interagire correttamente con il ciclo di gioco: deve partire e fermarsi nei momenti opportuni, cambiare il giocatore attivo a fine turno e determinare la conclusione della partita in caso di scadenza del tempo (TIMEOUT). Infine, essendo presenti funzionalità aggiuntive come il Replay, è necessario evitare incoerenze (ad esempio il tempo che continua a scorrere mentre si sta riproducendo una partita).

Soluzione La soluzione isola la gestione del tempo in un componente dedicato (**GameTimer**), separato sia dalla scacchiera sia dalla UI. Il timer esegue un decremento periodico del tempo del giocatore attivo e notifica gli aggiornamenti tramite callback agli osservatori del match, che a loro volta aggiornano la view. Dal punto di vista progettuale, la comunicazione verso l'esterno segue il pattern **Observer**: il match agisce da soggetto osservabile e propaga gli update di tempo ai subscriber (**ChessMatchObserver**) senza accoppiare direttamente il timer alla UI. In questo modo il timer rimane riutilizzabile e testabile, mentre la UI riceve soltanto eventi di aggiornamento.

In caso di scadenza del tempo, il timer invoca la callback di timeout e il match aggiorna lo stato della partita a **TIMEOUT** dichiarando un esito consistente. Per garantire coerenza con le funzionalità estese, in modalità Replay il timer viene sospeso e ripristinato solo quando si ritorna alla partita live e solo se la partita non è già terminata. Questo evita condizioni di race e mantiene allineati stato di gioco, visualizzazione e avanzamento temporale.

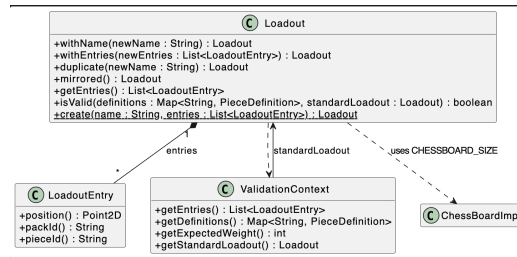


Figura 2.12: Loadout

Loadout

Problema L'applicativo supporta la compatibilità con **pezzi custom** e, di conseguenza, il giocatore deve poterli selezionare e utilizzare tramite i **loadout**. Tuttavia, la personalizzazione non può essere completamente libera: un **loadout** arbitrario rischierebbe di introdurre configurazioni *sbilanciate* (vantaggi ingiustificati), *inconsistenti* (assenza di pezzi fondamentali come il Re) oppure *non valide* dal punto di vista strutturale (più pezzi nella stessa posizione o posizioni fuori dall'area prevista). Inoltre, poiché i pezzi possono provenire da risorse esterne/mod, è necessario considerare anche il caso in cui un pezzo selezionato non sia più disponibile a runtime (ad esempio perché una mod è stata rimossa o aggiornata), evitando di avviare partite o salvare configurazioni che non siano ricostruibili. Il problema, quindi, è consentire l'uso di pezzi custom mantenendo al contempo **coerenza e bilanciamento** rispetto al **loadout standard**, che rappresenta il riferimento “corretto” per numero di pezzi, posizioni ammesse e valore complessivo.

Soluzione La soluzione consiste nell'introdurre una fase di **validazione del loadout** che confronta la configurazione scelta dal giocatore con un **loadout standard** di riferimento e con l'insieme delle **definizioni effettivamente disponibili** a runtime (inclusendo anche pezzi provenienti da mod esterne). Dal punto di vista progettuale, la validazione è implementata tramite una **pipeline di vincoli** basata su **Predicate** (approccio riconducibile al pattern **Specification**): ogni regola di validità è una specifica autonoma (es. peso totale, numero di entry, presenza del Re, posizioni distinte, esistenza delle definizioni, match delle posizioni e dei pesi rispetto allo standard) e tutte le specifiche vengono composte ed eseguite in sequenza con short-circuit sulla prima violazione. Il controllo si appoggia ad un *contesto di validazione* che incapsula i dati necessari (entries del loadout, mappa delle definizioni disponibili, peso atteso e loadout standard), rendendo il sistema estendibi-

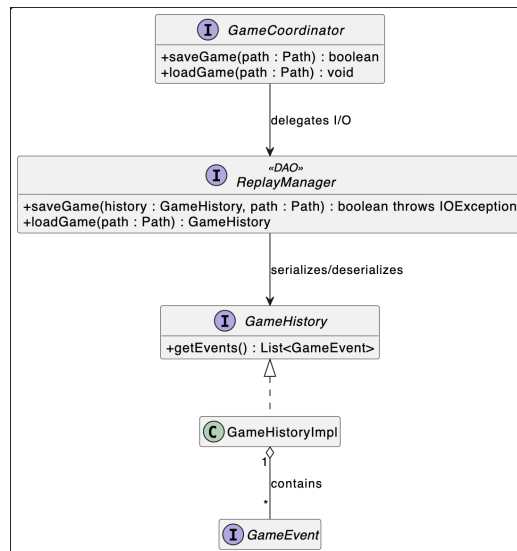


Figura 2.13: Save and load

le: aggiungere un nuovo vincolo richiede solo l'introduzione di una nuova specifica e la sua inclusione nella pipeline.

Inoltre, per garantire un uso corretto lato applicativo, la persistenza o l'attivazione di un loadout non standard viene subordinata al superamento della validazione: in questo modo si evita che la UI o la logica di gioco debbano gestire casi anomali a posteriori. La validazione delle definizioni, infine, garantisce robustezza anche con pezzi custom: se un pezzo non è presente tra le definizioni caricate, il loadout viene considerato non valido, prevenendo configurazioni non ricostruibili o incoerenti.

Salvataggio e caricamento

Problema L'applicativo deve consentire all'utente di **salvare** una partita in corso e di **caricarla** successivamente, ripristinando uno stato coerente in termini di posizionamento delle entità (anche pezzi custom), turno corrente, punteggio e tempo rimanente. Il salvataggio non deve dipendere dalla UI né dalla rappresentazione grafica, e deve restare compatibile con l'architettura generale (MVC) evitando che la scacchiera o i controller di view si occupino direttamente di I/O su file. Inoltre, poiché la partita integra anche Replay e logging, il salvataggio deve preservare una rappresentazione sufficientemente ricca da permettere la ricostruzione dello stato e la consultazione della cronologia.

Soluzione La soluzione si basa sulla serializzazione di una struttura dedicata (**GameHistory**), che contiene:

- la sequenza di eventi (**GameEvent**) necessaria per ricostruire lo stato della partita;
- i metadati indispensabili per ripristinare correttamente l'esperienza di gioco (loadout, tempo rimanente e punteggi).

Dal punto di vista dei design pattern, il flusso di salvataggio/caricamento viene orchestrato da un componente centrale che svolge il ruolo di **Coordinator/Facade** a livello applicativo: si occupa di coordinare i passi necessari (scelta percorso, invocazione persistenza, ripristino stato, aggiornamento della scena) mantenendo separati UI e Model. La persistenza è incapsulata in un componente dedicato (**ReplayManager**) che svolge il ruolo di **DAO**: espone un'interfaccia stabile per **saveGame/loadGame** e nasconde i dettagli di serializzazione JSON.

Al caricamento, la cronologia viene deserializzata e lo stato corrente viene ricostruito applicando gli eventi fino alla fine: questo permette di ripristinare la scacchiera senza assumere regole fisse e garantisce compatibilità con entità custom, poiché gli eventi trasportano le informazioni necessarie (ad esempio le entità coinvolte in spawn/despawn e i dati di cattura/promozione nei move). In questo modo si ottiene un sistema di save/load robusto, modulare e coerente con replay e logging, con responsabilità ben separate tra orchestrazione applicativa e persistenza.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

Si è deciso di sottoporre larga parte del codice a testing automatizzato in modo da verificarne la sopravvivenza ad aggiornamenti e refactor che sono stati fatti (o che potrebbero essere fatti in futuro). Si è scelto di non testare view e controller, ma di dare più importanza al testing del model che risulta essere la parte nevralgica dell'applicazione. Per il testing automatizzato è stata utilizzata la libreria JUnit. Gli elementi testati comprendono:

- Regole di Movimento
- Caricamento dei file di risorse esterne
- Istanziamento delle entità
- Regole avanzate di gioco
- Gestione dei turni
- Replay della partita
- Punteggio
- Loadout

3.2 Note di sviluppo

3.2.1 Giacomo Fantini

- Utilizzo della libreria SLF4J Utilizzata in vari punti. Qualche esempio è <https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac34>

```
src/main/java/it/unibo/turbochess/controller/loadercontroller/  
impl/LoaderControllerImpl.java#L122, https://github.com/Turbo-Chess/  
Turbo-Chess/blame/37711e8ef9c047229ac19adac348074fdc8f02c6/  
src/main/java/it/unibo/turbochess/controller/loadercontroller/  
impl/LoaderControllerImpl.java#L108
```

- Utilizzo di BiMap dalla libreria Google Guava. Permalink: <https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac348074fdc8f02c6/src/main/java/it/unibo/turbochess/model/chessboard/board/impl/ChessBoardImpl.java#L77-L79>, <https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac348074fdc8f02c6/src/main/java/it/unibo/turbochess/model/chessboard/board/impl/ChessBoardImpl.java#L45>
- Utilizzo della libreria Jackson. Qualche esempio: <https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac348074fdc8f02c6/src/main/java/it/unibo/turbochess/model/loader/impl/EntityLoaderImpl.java#L55>, <https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac348074fdc8f02c6/src/main/java/it/unibo/turbochess/model/entity/definition/AbstractEntityDefinition.java#L37-L46>, <https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac348074fdc8f02c6/src/main/java/it/unibo/turbochess/model/entity/definition/AbstractEntityDefinition.java#L117>
- Utilizzo di Stream e Lambda Expression Usate più volte, ad esempio: <https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac348074fdc8f02c6/src/main/java/it/unibo/turbochess/model/movement/impl/MoveRulesImpl.java#L80-L91>, <https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac348074fdc8f02c6/src/main/java/it/unibo/turbochess/model/loader/impl/EntityLoaderImpl.java#L36-L41>
- Utilizzo di method reference. Usate più volte, ad esempio: <https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac348074fdc8f02c6/src/main/java/it/unibo/turbochess/model/loader/impl/EntityLoaderImpl.java#L37>, <https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac348074fdc8f02c6/src/main/java/it/unibo/turbochess/model/loader/impl/EntityLoaderImpl.java#L40>, <https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac348074fdc8f02c6/src/main/java/it/unibo/turbochess/controller/loadercontroller/impl/LoaderControllerImpl.java#L106>
- Utilizzo di Optional: Utilizzati spesso, ad esempio: <https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac348074fdc8f02c6/>

`src/main/java/it/unibo/turbochess/model/chessboard/board/impl/ChessBoardImpl.java#L69-L71`

- Scrittura di builder generico. <https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac348074fdc8f02c6/src/main/java/it/unibo/turbochess/model/entity/impl/AbstractEntity.java#L133>
- Utilizzo di switch expressions Permalink: <https://github.com/Turbo-Chess/Turbo-Chess/blob/8457ae5852d122853804a2393f4a660afe3c7743/src/main/java/it/unibo/turbochess/model/movement/impl/MoveRulesImpl.java#L73-L77>

3.2.2 Brando Fabbri

- Utilizzo dei Logger, LoggerFactory dalla libreria SLF4J Utilizzata in alcuni punti. Per esempio: <https://github.com/Turbo-Chess/Turbo-Chess/blob/8457ae5852d122853804a2393f4a660afe3c7743/src/main/java/it/unibo/turbochess/model/rules/CheckCalculator.java#L45-L64>
- Utilizzo indiretto di BiMap dalla libreria Google Guava. Unico esempio concreto: <https://github.com/Turbo-Chess/Turbo-Chess/blob/8457ae5852d122853804a2393f4a660afe3c7743/src/main/java/it/unibo/turbochess/model/utils/RulesUtils.java#L55-L61>
- Utilizzo di Lambda expressions. Usate un po' di volte, come: <https://github.com/Turbo-Chess/Turbo-Chess/blob/8457ae5852d122853804a2393f4a660afe3c7743/src/main/java/it/unibo/turbochess/model/utils/RulesUtils.java#L106-L109>
- Utilizzo di Stream. Usate abbastanza volte, ad esempio: <https://github.com/Turbo-Chess/Turbo-Chess/blob/8457ae5852d122853804a2393f4a660afe3c7743/src/main/java/it/unibo/turbochess/model/utils/RulesUtils.java#L126-L130>
- Utilizzo di Optional: Utilizzati molteplici volte, ad esempio: <https://github.com/Turbo-Chess/Turbo-Chess/blob/8457ae5852d122853804a2393f4a660afe3c7743/src/main/java/it/unibo/turbochess/model/rules/AdvancedRules.java#L104-L117>, oppure <https://github.com/Turbo-Chess/Turbo-Chess/blob/8457ae5852d122853804a2393f4a660afe3c7743/src/main/java/it/unibo/turbochess/model/rules/CheckCalculator.java#L45-L57>

- Utilizzo di Switch expressions: utilizzati un paio di volte, ad esempio: <https://github.com/Turbo-Chess/Turbo-Chess/blob/8457ae5852d122853804a23src/main/java/it/unibo/turbochess/model/handler/impl/TurnHandlerImpl.java#L90-L94>

3.2.3 Luca Bevilacqua

- Utilizzo della libreria SLF4J Utilizzata in vari punti. Per esempio:
<https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac348074fdc8f02c6/src/main/java/it/unibo/turbochess/model/replay/impl/ReplayManagerImpl.java#L20>
- Utilizzo della libreria Lombok Usata più volte, come: <https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac348074fdc8f02c6/src/main/java/it/unibo/turbochess/model/replay/impl/GameHistoryImpl.java#L25>
- Utilizzo della libreria Jackson. Utilizzata più volte come: <https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac348074fdc8f02c6/src/main/java/it/unibo/turbochess/model/replay/impl/MoveEvent.java#L26>
- Utilizzo di Concorrenza / Scheduling . Usato per il timer: <https://github.com/Turbo-Chess/Turbo-Chess/blob/37711e8ef9c047229ac19adac348074fdc8f02c6/src/main/java/it/unibo/turbochess/model/timer/impl/GameTimerImpl.java#L40>
- Utilizzo di Stream. Usate abbastanza volte, ad esempio: <https://github.com/Turbo-Chess/Turbo-Chess/blob/7bc875a7f7dad03ddd7970d9058c16a32b9src/main/java/it/unibo/turbochess/model/loadout/impl/LoadoutImpl.java#L271>
- Utilizzo di Predicate: Utilizzati molteplici volte, ad esempio: <https://github.com/Turbo-Chess/Turbo-Chess/blob/7bc875a7f7dad03ddd7970d9058c16a32b9src/main/java/it/unibo/turbochess/model/loadout/impl/LoadoutImpl.java#L140>

Capitolo 4

Commenti finali

4.0.1 Giacomo Fantini

Mi sono occupato principalmente della parte riguardante la gestione delle entità, dei movimenti e del caricamento di entità custom da file prodotti dall'utente. Dal punto di vista progettuale sono abbastanza sicuro di aver svolto un discreto lavoro, cercando di realizzare una struttura il più pulita e generalizzabile possibile in modo da lasciare aperta opzione per futuri sviluppi. Il piano prevedeva di introdurre anche un altro tipo di entità, i "power ups" ma a causa dell'abbandono di un compagno di team, e quindi della redistribuzione dei compiti, ciò non è stato possibile. Ho comunque provveduto a creare una struttura che in futuro permetta l'aggiunta di questo tipo di features. Per quanto riguarda il caricamento la sfida più dura è stata accertarmi di gestire il tutto in modo che sia utilizzabile dai 3 sistemi operativi principali e dal classpath allo stesso tempo. Inoltre una parte cruciale è stata la necessità di verificare gli input e lanciare eccezioni in caso i dati inseriti dall'utente non siano conformi alle specifiche, cercando di dare le informazioni il più precise possibili per facilitarne la risoluzione. Sono state necessarie varie modifiche nel corso del tempo a causa di problemi scoperti durante il testing (come problemi con windows, problemi a caricare da classpath) e probabilmente questa potrebbe essere riscritto in modo migliore e più conciso.

4.0.2 Brando Fabbri

Il mio lavoro è stato quello di progettare il Gestore dei Turni e le Regole avanzate. La scelta di rendere i metodi delle regole statici si è rivelata ottimale sia in quel dato istante, sia verso il completamento del progetto, anche se ritengo che alcuni metodi potevano essere parzialmente ottimizzati; del Gestore dei Turni mi ritengo piuttosto soddisfatto, sono riuscito a scegliere

la convenzione migliore per gestire la comunicazione delle mosse disponibili con il Controller fin da subito, ma mi accorgo che piccole ottimizzazioni potevano essere fatte per gestire in modo più pulito il flusso del gioco. A livello architetturale ritengo di aver fatto un buon lavoro, a favore del fatto che sono riuscito più volte a individuare l'origine di vari bug senza dover debuggare. Come considerazioni finali ritengo di aver fatto un lavoro discretamente buono, ho fatto in modo che il mio codice fosse facilmente riutilizzabile in futuro, poiché l'idea di riprendere questo progetto ed estenderlo ulteriormente per nostro conto ci è passata per la testa più di una volta.

4.0.3 Luca Bevilacqua

Mi sono occupato principalmente delle funzionalità di qualità della vita e della gestione dello stato della partita: salvataggio/caricamento, Replay e log delle mosse, timer di partita, oltre a parte della logica collegata alla validazione dei loadout. Ritengo che la scelta di basare persistenza e Replay su una rappresentazione ad eventi sia stata efficace, perché consente di riutilizzare la stessa struttura sia per la serializzazione su file che per la navigazione della partita, mantenendo separazione tra Model e View.

Un punto di forza del lavoro è stato integrare controlli dedicati per l'utente senza introdurre dipendenze eccessive, sfruttando meccanismi di notifica (observer/callback) per aggiornare la UI. Come aspetto migliorabile, alcune parti potrebbero essere ulteriormente uniformate per rendere ancora più coerente l'organizzazione tra modalità "live" e modalità Replay e per ridurre potenziali duplicazioni di logica.

In caso di estensione del progetto, l'evoluzione naturale prevedrebbe l'arricchimento dei salvataggi tramite metadati (nome sessione, riepilogo, esito) e il potenziamento del sistema di Replay (highlight dell'ultima mossa, shortcut da tastiera). Tali implementazioni manterrebbero la coerenza con la logica dei turni e le regole esistenti, integrando inoltre il sistema dei power-up nel salvaggio e nel replay.

4.1 Difficoltà incontrate e commenti per i docenti

4.1.1 Giacomo Fantini

Questa è stata la mia prima esperienza di gruppo con un progetto di queste dimensioni. Una delle sfide più complicate è stata sicuramente la coordinazione con gli altri compagni di team, soprattutto la suddivisione del lavoro e

il riuscire a mettersi d'accordo su come ognuno avrebbe sviluppato le varie feature in modo da far integrare tutto nel migliore dei modi. Inoltre una cosa che ho trovato molto complicata è stato cambiare il modo di ragionare verso soluzioni orientate agli oggetti, dato che sono sempre stato abituato a ragionare in maniera più "C-Like".

4.1.2 Brando Fabbri

Sicuramente questa è stata la mia prima grande esperienza lavorativa di gruppo, viste le dimensioni del progetto è spesso stato ottimale coordinarsi con gli altri per programmare certi sistemi che poi l'altro avrebbe usato subito dopo. Il livello di dettaglio che ci era richiesto mi ha insegnato ad avere un certo tipo di approccio col progetto a favore anche degli altri compagni di gruppo, in un modo che non sarebbe fattibile facendo semplici esercitazioni seppur nello stesso ambito.

4.1.3 Luca Bevilacqua

Nonostante la mia esperienza lavorativa si svolga in una piccola realtà di sviluppo software, questo percorso mi ha permesso di apprezzare il valore concreto delle best practice di project management. Alla luce di ciò, consiglieri di dedicare ancora più spazio a questa disciplina nel programma, poiché la capacità di gestire il progetto è fondamentale per garantire ordine e qualità, a prescindere dalla dimensione dell'azienda.

Capitolo 5

Guida utente

5.0.1 Avvio del gioco

Per eseguire il gioco basta eseguire il comando "java -jar OOP25-turbochess.jar"

5.0.2 Organizzazione cartella risorse

Le risorse esterne del gioco sono raggruppate in una cartella situata in una posizione diversa in base al sistema operativo:

- Windows: C:\Users\user\AppData\Roaming\TurboChess\
- MacOS: ~/Library/Application\Support/TurboChess
- Linux: ~/.config/turbochess

All'interno di questa cartella si trovano altre 4 cartelle

- **Mods** Cartella che contiene i file json che definiscono un nuovo pezzo custom
- **Assets** Cartella che contiene le immagini dei pezzi da visualizzare
- **saves** Cartella che contiene i salvataggi delle partite
- **loadouts** Cartella che contiene i loadout dei pezzi per giocare una partita

5.0.3 Creazione di un pezzo custom

Scrittura del Json

Un pezzo custom viene definito da un file con estensione ".json" che ha questa forma:

```
{
  "type" : "PIECE",
  "name" : "Test",
  "id" : "test",
  "imagePath" : "TestPack",
  "pieceType" : "INFERIOR",
  "weight" : 3,
  "moveRules" : [ {
    "direction" : {
      "x" : 1,
      "y" : 1
    },
    "restriction" : "MOVE_AND_EAT",
    "hasMoved" : "false",
    "moveStrategy" : "SLIDING"
  } ]
}
```

- **type:** è il tipo di entità che si sta creando. Attualmente solo "PIECE" è supportato.
- **name:** è il "friendly-name" del pezzo che verrà visualizzato per esempio nel log delle mosse. Può essere una qualunque stringa. NON può essere vuoto.
- **id:** è l'identificatore del pezzo che viene usato internamente nel programma. Viene utilizzato anche per riferirsi alle immagini. Può essere una qualunque stringa. NON può essere vuoto.
- **imagePath:** è la relative path dell'immagine da aggiungere, a partire dalla cartella di risorse "Assets". Per esempio, ipotizzando di essere su linux e ipotizzando di volere inserire le immagini nella cartella "TestPack", nel file json basterà scrivere "TestPack". Questo verrà poi risolto in `~/.config/turbochess/Assets/TestPack`.
- **pieceType:** Rappresenta il tipo del pezzo da creare. Accetta i seguenti valori:
 - PAWN
 - INFERIOR
 - TOWER

- SUPERIOR
- KING

Dal `piecetype` si ereditano proprietà specifiche di alcuni pezzi ma NON il loro movimento. Per esempio un pezzo di tipo "pawn" può fare promozione, un pezzo di tipo "tower" può fare arrocco.

- **weighth**: rappresenta il peso di un pezzo sulla scacchiera. Questo valore influenza il punteggio e viene usato per applicare delle restrizioni ai loadout. Deve essere un valore maggiore di 0.
- **moveRules**: è una lista che rappresenta le possibili mosse del pezzo. Formato da 4 campi:
 - **direction** Vettore bi-dimensionale che specifica dove un pezzo può andare.
 - **restriction** Un movimento può essere ristretto a 3 specifici casi:
 - * **MOVE_ONLY** Un pezzo può andare in quella casella solo se libera
 - * **EAT_ONLY** Un pezzo può andare in quella casella solo se occupata da un pezzo nemico
 - * **MOVE_AND_EAT** Un pezzo può andare in quella casella sia se libera sia se occupata da un pezzo nemico
 - **hasMoved** Accetta due valori:
 - * **TRUE** Un pezzo può eseguire quella mossa solo se NON si è già mosso (es. il doppio passo del pedone)
 - * **FALSE** Un pezzo può eseguire quella mossa indipendentemente da se sia mosso oppure no
 - **moveStrategy** Rappresenta il tipo di movimento del pezzo. Accetta i seguenti valori:
 - * **SLIDING** Il pezzo si muove nella direzione indicata finché non incontra un ostacolo (es. un alfiere)
 - * **JUMPING** Il pezzo va nella casella specificata saltando eventuali nemici sul percorso (es. un cavallo)
 - * **STEPPING** Il pezzo va nella casella specificata senza saltare eventuali nemici sul percorso (es. pedone)

Viene fornito un esempio nel repository github sotto il nome di "EsempioPezziCustom".

5.0.4 Funzionamento del Movimento

Vettori di movimento

Un vettore indica la direzione in cui si può muovere una pedina, è formato da due coordinate: x ed y. Il piano cartesiano su cui si calcolano i movimenti è considerato con l'origine in alto a sinistra, per cui le direzioni sono da calcolare tenendo conto di ciò. Quando si crea un movimento custom le direzioni si considerano a partire dal giocatore **nero**, il gioco gestisce in automatico l'inversione del vettore y quando si calcola il movimento per il giocatore bianco. Di seguito un elenco dei vettori principali da usare come esempio per tutti gli altri:

- Avanti: $(0, 1)$
- Indietro: $(0, -1)$
- Destra: $(1, 0)$
- Sinistra: $(-1, 0)$
- Diagonale avanti a sinistra: $(-1, 1)$
- Diagonale indietro a sinistra: $(-1, -1)$
- Diagonale avanti a destra: $(1, 1)$
- Diagonale indietro a destra: $(1, -1)$
- **STEPPING** Il movimento **stepping** è un tipo di movimento dove la pedina si muove in una certa posizione solamente se la strada è totalmente libera. In caso di movimento diagonale la strada viene considerata percorsa prima sull'asse x e poi sull'asse y.
- **SLIDING** Il movimento **sliding** è un tipo di movimento dove il pezzo si muove lungo la direzione indicata finchè non incontra un ostacolo.
- **JUMPING** Il movimento **jumping** è un tipo di movimento dove la pedina si muove nella cella indicata dal suo vettore di movimento saltando eventuali ostacoli sulla scacchiera.

Disposizione corretta dei file

- **JSON** I file ".json" vanno inseriti all'interno di una cartella dentro "Mods". Creare una cartella è cruciale in quanto il suo nome sarà automaticamente il nome del modpack da cui provengono tutti i pezzi al suo interno.
- **Immagini** I file immagine DEVONO per forza essere nel formato ".png". Il nome dei file immagine deve essere esattamente "white_id.png" e "black_id.png" dove "id" è il campo id assegnato al pezzo nel relativo file ".json". Le immagini vanno inserite all'interno della propria cartella che sarà situata a sua volta nella cartella default "Assets".

Una volta aver creato il file json ed aver individuato le immagini, sarà necessario inserire la cartella con i file json all'interno di "Mods" e la cartella con le immagini all'interno della cartella "Assets". Fatto ciò i pezzi saranno accessibili tramite la schermata di loadout.

5.0.5

5.0.6 Regole Avanzate

Questa è una sezione che spiega le regole avanzate di scacchi, a ruolo puramente informativo, che comprendono:

- **Scacco:** Quando ad un pezzo, dopo essersi spostato, gli si presenta la possibilità di mangiare il re avversario nel prossimo turno, si dice che tale giocatore sta mettendo sotto scacco tale re. Ci sono 3 modi per difendersi da questo:
 - Spostare il re, se possibile,
 - Mettere un proprio pezzo in mezzo alla linea d'attacco imposta sul re, se possibile e se il pezzo non salta,
 - Mangiare il pezzo attaccante.
- **Doppio Scacco:** Il doppio scacco è un caso particolare in cui, muovendo un proprio pezzo per mettere sotto scacco il re, si libera un altro proprio pezzo che a sua volta mette sotto scacco il re. L'unico e solo modo per evitare l'attacco è muovere il re.
- **Scacco Matto:** Nel caso di Scacco o Doppio scacco, se NESSUNA delle opzione sopra elencate è realizzabile, significa che si è appena subito uno Scacco matto, ovvero l'impossibilità di difendere il re da un attacco, evento che segna il fine partita in sconfitta.

- **Patta:** La patta è la situazione in cui la partita finisce senza un vero vincitore e può essere causata da 2 situazioni:
 - per Stallo: ovvero il re non è sotto scacco ma nessuna mossa è possibile con alcun pezzo,
 - per Impossibilità pratica di Scacco Matto: anche se più informale (e dipende dalle usanze) quando sulla scacchiera rimangono solo i due re di colore opposto, oppure i due re e uno dei due giocatori dispone di un singolo pezzo di tipo inferiore (come Alfiere o Cavallo), visto che sarebbe comunque impossibile causare uno scacco matto all'avversario.
- **Arrocco:** L'arrocco è una mossa speciale difensiva che permette di spostare il re con una delle due torre a patto che:
 - Il re non sia mai stato mosso,
 - La o le torri non siano mai state mosse,
 - Le celle tra il re e le torri siano libere,
 - Le celle tra il re e le torri non siano sotto attacco.

Allora sarà possibile eseguire un arrocco cliccando prima il re per poi vedere la nuova mossa disponibile.

- **Promozione:** Nel caso specifico in cui un pedone raggiunge il lato opposto della scacchiera rispetto al proprio campo, si ha diritto a promuoverlo in un qualsiasi pezzo della propria collezione iniziale (pedoni e re esclusi).

5.0.7 Creazione di un Loadout

Nella schermata di `LoadoutEditor` è possibile cliccare su un qualsiasi pezzo che compare nella lista di sinistra, non appena selezionato, cliccando su un qualsiasi bottone della griglia quel pezzo verrà posto in quell'esatto punto del loadout. Il programma non controlla sul momento la validità del loadout (poiché dovrebbe analizzarlo comunque nella sua interezza), infatti elencherò qui sotto le regole necessarie in modo che un loadout sia considerato valido:

- Le torri vanno messe esclusivamente negli angoli della riga in basso a sinistra e destra,
- La regina (o qualsiasi pezzo Superior) va posta nella quarta cella nella riga in basso,

- Il re va (o qualsiasi pezzo King) posto nella quinta cella nella riga in basso,
- La riga in alto è composta solo da pezzi di tipo Pawn,
- Il peso di tutti i pezzi è quello che ci si aspetterebbe, e quindi identico al loadout standard,
- Il nome inserito nel TextLabel non sia vuoto o composto solo da spazi.

Fatto ciò sarà possibile cliccare il tasto Save per salvarlo in memoria e riutilizzarlo più in avanti.

5.0.8 Visualizzare e usare un Loadout

Nella schermata di `LoadoutSelector` verranno visualizzati a sinistra tutti i loadout salvati in memoria e sarà possibile cliccare per selezionarli. Da selezionato si può scegliere se, caricare il loadout (Load) per vedere un elenco di tutti i pezzi che compongono quel loadout, oppure se usare il loadout (Use), in cui bisognerà però scegliere per quale giocatore, semplicemente spuntando o meno la CheckBox "For Black", ovvero se usarlo per il nero o per il bianco.

Capitolo 6

Esercitazioni di laboratorio

6.0.1 giacomo.fantini5@studio.unibo.it

- Laboratorio 07: <https://virtuale.unibo.it/mod/forum/discuss.php?d=12345#p123456>
- Laboratorio 08: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p286162>
- Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287266>

6.0.2 luca.bevilacqua6@studio.unibo.it

- Laboratorio 07: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207193#p284623>
- Laboratorio 08: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p285388>
- Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287275>
- Laboratorio 11: <https://virtuale.unibo.it/mod/forum/discuss.php?d=210617#p288851>
- Laboratorio 12: <https://virtuale.unibo.it/mod/forum/discuss.php?d=211539#p290764>